# Selecting the Category-Count Tilt in the FFT-Tree Multinomial Series

A saddlepoint replacement for the endpoint-chord heuristic

**Abstract**

The FFT-tree evaluator used throughout this codebase to compute the multinomial characteristic function reconstructs a single target coefficient, $[u^N]$, from a hierarchical CRT-style ring-splitting recursion. Every merge step in that recursion aliases many coefficients together under one shared floating-point exponent, so the computation is only numerically safe when the polynomial being merged has its probability mass concentrated near the coefficient actually being extracted. Each per-category factor may be rescaled by $\exp(\nu\, i)$ before entering the recursion, exactly and for free, so long as the rescaling of every later factor is undone by a matching correction — this document works out exactly which rescalings are legal, shows that the rescaling previously in use (an endpoint chord) drives the recursion's intermediate mass in the wrong direction as problem size grows, and derives the correct choice: the unique tilt $\nu^\star$ that makes the count $N$ the *mean*, rather than a tail value, of the tilted joint distribution — the classical saddlepoint/exponential-tilting construction, adapted here to a coefficient-extraction setting where the "sample size" $N$ is fixed by the problem rather than drawn at random. We give the derivation, prove the compensability lemma that pins down why only a single global tilt is admissible, describe the fixed-iteration safeguarded Newton–bisection solver used to compute it (once inside the gamma search, then reused as a stored constant by every series evaluator), and report the resulting accuracy gain.

## Contents

**1 Setup and notation** **1**

**2 The exact rescaling freedom** **2**
2.1 Why a rescaling is needed at all . . . 2
2.2 Exact compensation via delta-rescaling . . . 2
2.3 Only an exponential-in-index rescaling is admissible . . . 3

**3 Why the endpoint chord fails** **3**

**4 The saddlepoint (mean-matching) rate** **4**
4.1 Derivation . . . 4
4.2 Relation to Hayman's method and the saddle-point method in analytic combinatorics 4
4.3 Why a single global tilt, and not one per round . . . 5

**5 Numerical solution** **5**
5.1 The naive approach and why it fails . . . 5
5.2 Safeguarded Newton–bisection . . . 5

**6 Reuse in the gamma search** **6**

**7 Empirical validation** **6**

**8 A known limitation at extreme problem sizes** **7**

# 1 Setup and notation

Let $K$ be the number of categories and $N$ the total count. For category $k$ and count $i \in \{0, \dots, N\}$, the code stores two per-cell quantities in `matrix[k*(N+1)+i]`:

- `log_prob`: $\log p_k(i) = i \log p_k - \log i!$, the log-probability of drawing $i$ items in category $k$;
- `reward`: $a_k(i)$, the reward contribution of category $k$ at count $i$, centered so that $\sum_k a_k(x_{0,k}) = 0$ at the observed configuration $x_0$.

The transform the FFT tree evaluates is

$$M_S(z) \;=\; \mathbb{E}\big[e^{-zS}\big] \;=\; N!\,[u^N] \prod_{k=0}^{K-1} G_k(u; z), \qquad G_k(u; z) \;=\; \sum_{i=0}^{N} p_k(i)\, e^{-z\, a_k(i)}\, u^i,$$

where $S = \sum_k a_k(X_k)$ is the (centered) reward statistic and $z = \gamma + i\omega$ lies on a vertical Bromwich contour whose real offset $\gamma$ is chosen once, globally, by the gamma search (`fill_gamma`).

It is worth being precise about what the surrounding algorithm does with $M_S$, since it fixes the role of every parameter appearing below. The factor the evaluators multiply into each term, $\mathsf{sinc}(z) = (1 - e^{-zW})/(zW)$ with $W = \pi/T$, is exactly the transform of a uniform smoothing variable, $\mathsf{sinc}(z) = \mathbb{E}[e^{-zU}]$ for $U \sim \mathrm{Unif}[0, W]$ independent of $S$. Each stored term is therefore $M_S(z_n)\,\mathsf{sinc}(z_n) = M_{S+U}(z_n)$, and summing them by the trapezoid rule recovers the *density* of $S + U$ at the origin:

$$p \;=\; P(S \le 0) \;=\; W \cdot f_{S+U}(0),$$

exactly, because $f_{S+U}(0) = \frac{1}{W} P(-W \le S \le 0)$. Convolving with $U$ is what gives the atomic $S$ an honest density, spreading each point mass rightwards over a box of width $W$; requiring $W \ge -s_{\min}$ makes that box cover the event, and $W \ge s_{\max}$ kills the periodization image at $2W$, which together are `fill_interval`'s $W = \max(s_{\max}, -s_{\min})$. The truncation error of that sum, and the reason it behaves very differently for the log-likelihood-ratio and chi-squared statistics, are the subject of the companion note `latex/error_bounds.tex`; here $\gamma$ is simply held fixed and the question is how to condition the coefficient extraction that produces $M_S(z_n)$.

Write

$$w_k(i) \;=\; p_k(i)\, e^{-\gamma\, a_k(i)}$$

for the (real, positive) per-cell weight after the contour's real part $\gamma$ has been applied — this is exactly what `fill_cells` bakes into `log_prob` for the series evaluators, and what the gamma-search evaluators compute on the fly for each candidate $\gamma$. The object actually computed by the FFT tree is the polynomial product $\prod_k \big(\sum_i w_k(i)\, u^i\big)$, truncated to degree $N$ after every category (any partial count exceeding $N$ can never be reached again, since later categories only add non-negative amounts), from which the coefficient $[u^N]$ is read off at the end.

## 2 The exact rescaling freedom

### 2.1 Why a rescaling is needed at all

The FFT tree computes the product $\prod_k B_k(u)$ hierarchically: it factors the ring $\mathbb{Z}[x]/(x^M - \rho)$ into two half-size child rings at each level, tracking a single shared scalar $\rho$ (equivalently, one shared floating-point exponent `log_mod`) per block. This is only numerically safe when the values being merged under that one shared exponent are of comparable magnitude — if the coefficient the computation actually needs is many orders of magnitude below the "bulk" mass also folded into the same block, the merge step's subtraction (used to reconstruct one child ring's contribution from two aliased sums) loses more significant digits than the floating-point mantissa provides. Concretely: if $w_k(i)$ peaks far from $i \approx N/K$ in each round, the intermediate partial products have most of their mass sitting at indices that later get discarded by the $i > N$ truncation, and the surviving coefficient $[u^N]$ is reconstructed as a tiny difference between two much larger numbers.

### 2.2 Exact compensation via delta-rescaling

The code's existing (pre-fix) mechanism already exploits one degree of freedom to control this: scale category $k$'s factor by $\exp(\ell_k\, i)$ before combining it into the running product $A(u) = \prod_{k'>k} B_{k'}(u)$, and simultaneously rescale the *already-accumulated* $A$ by $\exp\big((\ell_k - \ell_{k-1})\, j\big)$ so that its scale is converted from the previous round's rate to the current one. This is exactly what the `d_k = l_k - l_prev` / `A[j].log_mod += d_k*j` lines do at every round. Tracing the recursion to the end shows this is not a reweighting of the answer at all: after the final round, $A(u) = \prod_k B_k\big(u\, e^{\ell_{K-1}}\big)$ picks up the same factor $e^{\ell_{K-1}N}$ on every composition, which the code removes exactly via `-l_prev*N` in the final combination step. All rates other than the last one cancel completely within the recursion. It is therefore a pure floating-point renormalization of intermediate arrays, with no effect on the value being computed — the same role played by the `shift` field used elsewhere in the codebase.

### 2.3 Only an exponential-in-index rescaling is admissible

**Lemma 1** (Compensability lemma)**.** *Suppose $A(u) = \sum_j a_j u^j$ and $B(u) = \sum_j b_j u^j$ are rescaled by $a_j \mapsto w(j)a_j$ and $b_j \mapsto v(j)b_j$ before being convolved, and the rescaling must be exactly undoable by multiplying the resulting coefficient $c_i = \sum_j a_{i-j}b_j$ by a correction depending only on $i$. Then $w$ and $v$ must be of the form $w(j) = w(0)\, r^j$, $v(j) = v(0)\, r^j$ for a common ratio $r$; no other choice of $w, v$ is compensable this way.*

*Proof.* The rescaled convolution's $i$-th coefficient is $\sum_j w(i-j)v(j)\, a_{i-j}b_j$. For this to equal $c_i$ times a function of $i$ alone (for arbitrary $a, b$), the coefficient $w(i-j)v(j)$ itself must depend only on $i$, not on the individual split $(i-j, j)$. Setting $j = 0$ gives $w(i)v(0) = f(i)$; setting $j = i$ gives $w(0)v(i) = f(i)$; combining, $w(i-j)v(j) = w(i)v(0) \Rightarrow w(i-j)v(j)/v(0) = w(i)$, and taking $j = 1$ against $i-1, i$ shows $w(i) = w(i-1) \cdot \big(v(1)/v(0)\big)$ for every $i$, i.e. $w$ satisfies the discrete multiplicative Cauchy equation $w(i) = w(i-1)\, r$ with $r := v(1)/v(0)$. Its only solutions are geometric, $w(i) = w(0)\, r^i$ (this is the standard uniqueness result for Cauchy's functional equation restricted to a discrete, totally ordered domain, where no regularity assumption — continuity, measurability, monotonicity — is even needed to rule out pathological solutions, unlike the real-domain case; see Aczél (1966), Ch. 2). Substituting back shows $v$ must share the same ratio $r$. □

In other words: the freedom the codebase already exploits (one geometric rate per convolution round) is not merely convenient, it is *all* the freedom that exists — there is no other choice of per-index rescaling, at any round, that leaves the final coefficient exactly unchanged. This closes off

two tempting-looking generalizations: a rescaling that depends on $i$ in some non-exponential way (e.g. to match a specific target profile), or a quadratic-in-index correction (explored and rejected earlier in this project's development, for the same algebraic reason: a quadratic term does not factor across the convolution sum the way a geometric one does). The only remaining question is which value of the rate $\ell_k$ (equivalently, $r = e^{\ell_k}$) to use at each round.

# 3 Why the endpoint chord fails

The rate previously used for $\ell_k$ was the endpoint chord,

$$\ell_k \;=\; -\,\frac{\log w_k(N) - \log w_k(0)}{N},$$

i.e. the slope that makes category $k$'s tilted profile $w_k(i)e^{\ell_k i}$ have equal value at $i = 0$ and $i = N$. For $w_k(i) = p_k(i)$ untilted (or with $\gamma$ folded in but not dominating), this chord makes each category's *own* profile peak near $i^\star = N/e$, independently of $p_k$: differentiating $\log w_k(i) = i \log p_k - \log i! + \ell_k i$ and using Stirling's approximation $\log i! \approx i \log i - i$ gives a stationary point where $\log p_k - \log i^\star + 1 + \ell_k = 0$, and substituting the chord's own definition of $\ell_k$ collapses the $p_k$-dependence, leaving $i^\star = N/e$ regardless of category. With $K$ categories processed sequentially, the *product*'s accumulated mass is therefore pulled toward roughly $(K/e)\,N \approx 0.368\,KN$: for $K \geq 3$ this already exceeds the target index $N$ ($1.10\,N$ at $K = 3$), and the excess grows linearly in $K$ — $1.84\,N$ at $K = 5$, $3.68\,N$ at $K = 10$, $5.52\,N$ at $K = 15$, tracking the three cases in Table 1 and their monotonically worsening error. This is precisely the "target far below the bulk mass" failure mode described in Section 2: the coefficient the tree needs to extract sits deep in the truncated tail of what each round is actually computing.

Empirically (Table 1), this is not a mild inefficiency but a correctness cliff: at $N = 100$ the chord is a few orders of magnitude worse than machine precision, but by $N = 1000$ it has degenerated into pure numerical noise (relative error $\sim 10^{19}$), and by $N = 5000$ the recovered coefficient carries no correct digits at all.

# 4 The saddlepoint (mean-matching) rate

## 4.1 Derivation

The natural replacement is the classical exponential tilt used to make a rare target value typical: choose the single scalar $\nu$ so that, under the tilted weights $w_k(i)\,e^{\nu i}$, the *mean* total count across all $K$ categories equals exactly $N$. Writing

$$m_k(\nu) \;=\; \frac{\sum_{i=0}^{N} i\, w_k(i)\, e^{\nu i}}{\sum_{i=0}^{N} w_k(i)\, e^{\nu i}}, \qquad \mathrm{Var}_k(\nu) \;=\; \frac{\sum_{i=0}^{N} i^2\, w_k(i)\, e^{\nu i}}{\sum_{i=0}^{N} w_k(i)\, e^{\nu i}} - m_k(\nu)^2,$$

$\nu^\star$ solves

$$\sum_{k=0}^{K-1} m_k(\nu) \;=\; N. \tag{1}$$

Since each $w_k(\cdot)e^{\nu\cdot}$ is (up to normalization) a one-parameter natural exponential family in $i$ with $i$ as its sufficient statistic, $m_k$ is its mean function and $dm_k/d\nu = \mathrm{Var}_k(\nu) \geq 0$; hence $\sum_k m_k$ is

non-decreasing in $\nu$, running continuously from 0 (as $\nu \to -\infty$, all mass at $i = 0$) to $KN$ (as $\nu \to +\infty$, all mass at $i = N$), so a root of (1) exists whenever $0 < N < KN$, i.e. for every problem with $K \geq 2$ (the $K = 1$ case is trivial: the single category's count is deterministically $N$).

### 4.2 Relation to Hayman's method and the saddle-point method in analytic combinatorics

Equation (1) is the coefficient-extraction saddle-point equation in its standard form: for a generating function $F(u) = \sum_n f_n u^n$ evaluated on a circle $u = \nu$ (here real and positive, i.e. $e^\nu$ in our multiplicative notation), the saddle point that makes the target coefficient's contribution dominate the contour integral is exactly the one solving $uF'(u)/F(u) = n$ — for a product of independent factors $F = \prod_k G_k$, $uF'/F = \sum_k uG_k'/G_k = \sum_k m_k(u)$, which is precisely (1). This is the discrete/coefficient-asymptotic form of Hayman's admissibility method (Hayman, 1956) and is treated in full generality (with this same mean-matching characterization of the saddle point) in the saddle-point chapter of Flajolet and Sedgewick (2009). On the statistics side, the identical construction — tilt a density so that a specified value becomes its mean, then approximate around that point — is Daniels's saddlepoint approximation (Daniels, 1954) and, at the level of the tilting operation itself, the Esscher transform (Esscher, 1932); it is also the large-deviations-optimal exponential change of measure described in Dembo and Zeitouni (1998), and its use for approximating tail probabilities of a sum via a saddlepoint equation of this exact shape is the basis of the Lugannani–Rice formula (Lugannani and Rice, 1980).

### 4.3 Why a single global tilt, and not one per round

Lemma 1 already shows a genuinely different rate at each round is mechanically permitted (nothing in the delta-rescale machinery requires $\ell_k$ to be constant across $k$ — the original chord code used a different value per category). The question is whether varying $\nu$ by round can do *better* than fixing it. Write $M_k(\nu) := \sum_{k' \geq k} m_{k'}(\nu)$ for the partial mean of categories $k, \ldots, K-1$ under a single global $\nu$; this is exactly the accumulated mean that $A(u)$'s profile carries after round $k$ under the constant tilt $\nu^\star$. Because $M_k$ is monotone in $k$ (increasing as $k \to 0$, i.e. as more categories are folded in) and satisfies $0 \leq M_k(\nu^\star) \leq N$ at every round by construction, every intermediate accumulator's mass is already inside the truncation window $[0, N]$ throughout the recursion — there is no round at which the accumulated profile overshoots into territory that later gets discarded. This is also the theoretically correct choice, not merely a convenient one: tilting every component of a sum of independent random variables by the *same* scalar is the standard recipe for shifting the sum's mean to a target value, and it is exactly this common tilt that gives each partial sum $\sum_{k' \geq k} X_{k'}$ its consistent conditional-mean structure given the total (a basic fact about natural exponential families: tilting does not change the conditional law of the summands given their total, and the first-order/Laplace approximation of that conditional mean under the correctly-tilted family is exactly $M_k(\nu^\star)$). A genuinely different tilt at each round would decouple the rounds from this coherent structure without a compensating benefit, and was found (Section 8) not to improve on the single global choice in practice.

# 5 Numerical solution

## 5.1 The naive approach and why it fails

Equation (1) is solved by Newton's method, $\nu \leftarrow \nu - f(\nu)/f'(\nu)$ with $f(\nu) = \sum_k m_k(\nu) - N$, $f'(\nu) = \sum_k \mathrm{Var}_k(\nu)$. A first implementation used a *fixed* number of Newton iterations with the step clamped to a fixed absolute bound, to keep the gamma-search evaluators — which solve (1) once per candidate $\gamma$, vectorized across SIMD lanes with a different $\gamma$ (hence a different target equation) per lane — free of any data-dependent loop-trip count. This fails for a structural reason, not merely an unlucky constant: since $f'(\nu) = \sum_k \mathrm{Var}_k(\nu)$ and $\mathrm{Var}_k$ is $O(N)$ for a natural exponential family over a range of size $N$ (exactly as the variance of a binomial-type distribution scales with its size), the transition width of $f$ around its root — the range of $\nu$ over which $f$ moves appreciably — shrinks like $O(1/N)$: a step size safe at $N = 1000$ already jumps clean past the transition at $N = 5000$–$10000$, landing in the fully saturated regime (all mass at $i = N$, $\mathrm{Var}_k = 0$ exactly), whose zero derivative sends the next step straight back past the origin — an infinite two-cycle that never converges, independent of how many iterations are budgeted.

## 5.2 Safeguarded Newton–bisection

The fix is the standard remedy for exactly this failure mode: maintain a bracket $[\ell, h]$ known to contain the root (any $\nu$ with $\pm B$ for a generously large constant $B$ works, since $\sum_k m_k$ is provably monotone from 0 to $KN$), and at each iteration take the Newton step only if it lands strictly inside the bracket *after* narrowing the bracket using the current sign of $f$; otherwise fall back to the bisection midpoint. This is the `rtsafe` algorithm of Press et al. (1992), guaranteeing convergence at least as fast as bisection (so a fixed iteration count is genuinely safe to pick in advance, unlike with plain Newton) while still enjoying Newton's quadratic convergence once the iterate is close enough. Because both the bracket update and the Newton-vs-bisection choice are ordinary comparisons, they compile to branchless `VEC_SELECT` blends rather than control-flow branches, so every SIMD lane performs identical work regardless of how far its particular $\gamma$'s root is from the starting point — the property the fixed-iteration design was after in the first place, now obtained safely.

# 6 Reuse in the gamma search

Because $w_k(i)$ after $\gamma$-tilting on the fly (during the gamma search, for a candidate $\gamma$) is algebraically identical to $w_k(i)$ after `fill_cells` bakes the winning $\gamma$ permanently into `log_prob`, the value of $\nu^\star$ computed for the *winning* $\gamma$ during the search is exactly the value the series evaluators need. Rather than have each of the six series evaluators (real, double-double, and quad precision, each with and without a precomputed merge-factor plan) re-derive $\nu^\star$ from scratch, the gamma search now stores it once, as `mult->l_rate`, alongside the existing `mult->gamma`; the series evaluators simply read it. This removes a redundant $O(KN)$-per-Newton-iteration solve from every series-evaluation call site, at the cost of one extra vectorized solve per candidate $\gamma$ during the search (itself cheap relative to the gamma search's own $O(KM \log M)$ FFT-tree evaluation per candidate).

# 7 Empirical validation

Accuracy is measured as the maximum, over the first few series terms, of the relative error against a direct $O(N^2K)$ reference evaluator that needs no rescaling trick at all (and is therefore immune

to the cancellation failure mode being fixed). All $K$ categories use $p_k \propto \sqrt{k+1}$; $M$ is the tree's padded size, $M = 2 \cdot 2^{\lceil \log_2(N+1) \rceil}$.

Table 1: Endpoint chord vs. saddlepoint tilt, real-precision series evaluator.

| Case | $N$ | $K$ | Chord (old) | Saddlepoint (new) |
|---|---|---|---|---|
| small | 100 | 5 | $7.5 \times 10^{-12}$ | $1.3 \times 10^{-14}$ |
| demo | 1000 | 10 | $1.6 \times 10^{19}$ | $2.4 \times 10^{-13}$ |
| large | 5000 | 15 | $2.9 \times 10^{86}$ | $3.8 \times 10^{-12}$ |

Table 2: Saddlepoint tilt, all six series evaluators (with/without precomputed plan give identical figures to the precision shown). The double-double column uses the `dd_cis`/`vdd_cis` twiddle factor of Section 8, not the plain-`double` cos / sin used earlier in this project's development.

| Case | $N$ | $K$ | Real | Double-double | Quad |
|---|---|---|---|---|---|
| small | 100 | 5 | $1.3 \times 10^{-14}$ | $1.1 \times 10^{-14}$ | $4.0 \times 10^{-15}$ |
| demo | 1000 | 10 | $2.9 \times 10^{-13}$ | $2.6 \times 10^{-13}$ | $2.4 \times 10^{-14}$ |
| large | 5000 | 15 | $2.5 \times 10^{-12}$ | $2.2 \times 10^{-12}$ | $6.0 \times 10^{-14}$ |

The residual error in Table 2 tracks a generic random-walk-of-rounding estimate, $\sqrt{n_{\text{ops}}}\,\varepsilon$ with $n_{\text{ops}} \approx KM \log_2 M$ and $\varepsilon$ the working type's machine epsilon, to within a small constant factor (well under an order of magnitude, 0.6–6×) across three orders of magnitude in $N$ — consistent with the systematic cancellation this construction targets having actually been removed, leaving only ordinary floating-point accumulation error. The double-double column sits consistently a little below the real column, as expected of the extra guard digits in its `unit` accumulator, though both remain bottlenecked by the same `log_mod` precision ceiling described in Section 8, which is why they are close rather than differing by many orders of magnitude the way double-double vs. real would for a computation genuinely limited by `unit`'s precision alone.

## 8 A known limitation at extreme problem sizes

Pushed to $N = 10^4$, $K = 20$ (well beyond anything in Table 2), the real- and double-double-precision evaluators both degrade to a relative error of $\sim 3.6 \times 10^{-4}$, while quad precision remains clean ($4.7 \times 10^{-13}$). Two candidate explanations were tested and ruled out: (i) the fixed-iteration solver failing to converge at this scale — refuted by comparing against an unrelated, fully adaptive bracket-and-Newton solver, which agrees with the fixed-iteration result to six significant figures and shows the identical residual error; (ii) the double-double evaluator's twiddle factor $\cos(\arg) + i\sin(\arg)$ having been computed in plain double precision rather than double-double, inconsistent with the double-double precision claimed elsewhere in that evaluator — this *was* a real inconsistency, since fixed by adding `dd_cis`/`vdd_cis`, double-double-precision cos / sin built the same way `dd_exp` already is (range-reduce modulo $2\pi$ in double-double precision, then a scaled Taylor series with repeated complex squaring) and now used throughout, including in Table 2; it was worth fixing on its own merits for consistency, but it does not explain this particular residual: the error at $N = 10^4$ is unchanged whether the twiddle factor is computed in double or double-double precision.

The actual cause is that `log_mod`, the shared floating-point exponent every merge step in the tree operates on, is tracked in plain `double` throughout the real- and double-double-precision evalu-

ators alike — only the double-double evaluator's magnitude/phase accumulator (`unit`) is genuinely double-double; the quad-precision evaluator, by contrast, tracks `log_mod` natively in `__float128`. Since `log_mod` is exactly the quantity every merge comparison and every call to the exponential correction factor depends on, its precision sets a floor under the whole computation that no amount of extra precision elsewhere — including the `dd_cis` fix above — can lift. Promoting `log_mod` to double-double throughout would close this gap but touches every arithmetic site in three files and is out of scope here.

## References


J. Aczél. *Lectures on Functional Equations and Their Applications*. Mathematics in Science and Engineering, vol. 19. Academic Press, New York, 1966.

H. E. Daniels. Saddlepoint approximations in statistics. *Annals of Mathematical Statistics*, 25(4):631–650, 1954.

A. Dembo and O. Zeitouni. *Large Deviations Techniques and Applications*, 2nd edition. Springer-Verlag, New York, 1998.

F. Esscher. On the probability function in the collective theory of risk. *Skandinavisk Aktuarietidskrift*, 15(3):175–195, 1932.

P. Flajolet and R. Sedgewick. *Analytic Combinatorics*. Cambridge University Press, 2009. Chapter VIII, "Saddle-Point Asymptotics."

W. K. Hayman. A generalisation of Stirling's formula. *Journal für die reine und angewandte Mathematik*, 196:67–95, 1956.

R. Lugannani and S. Rice. Saddle point approximation for the distribution of the sum of independent random variables. *Advances in Applied Probability*, 12(2):475–490, 1980.

W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. *Numerical Recipes in C: The Art of Scientific Computing*, 2nd edition. Cambridge University Press, 1992. §9.4, "Newton-Raphson Method Using Derivative" (the `rtsafe` routine).